CA1: LCR algorithm

Theory

LCR time complexity

In a basic ring network, given n nodes the LCR algorithm should be $O(n + 1)$ rounds and is the time it takes for the leader to travel around the ring network twice.

LCR space complexity

In a basic ring network, the LCR algorithm should produce worst case n^2 messages when the IDs are in ascending order.

Basic ring network tests

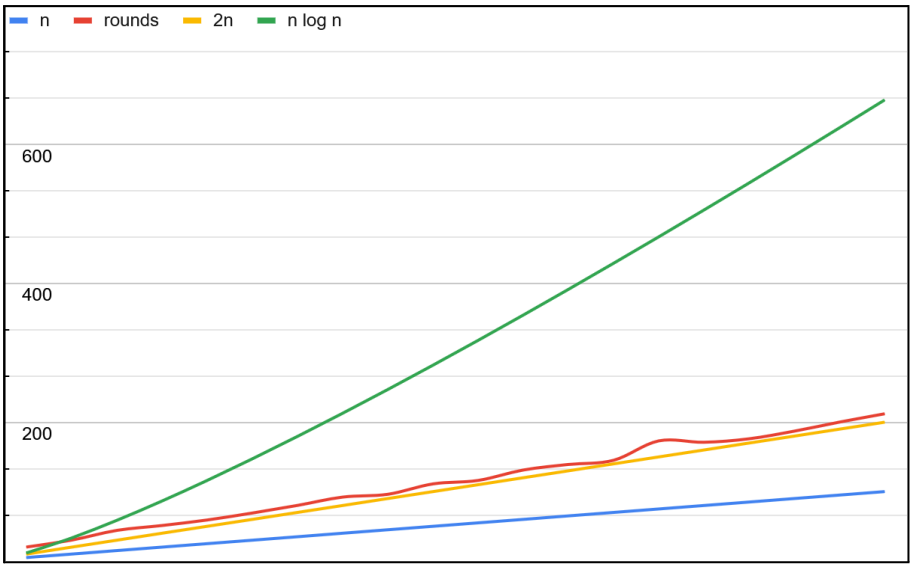
For the basic scheduler, only random, shuffled IDs were used. This means no ascending or descending perfect orders. Worst and best cases will be analysed in the ring of rings network, these tests exist merely to show that my implementation of the LCR algorithm follows what we expect in terms of running time and space complexity.

For $n = 5$ **Nodes** up to 100 in steps of 5. Some results have been hidden for brevity.

n	Rounds	Messages	$2n$	$n \log n$	n^2
10	30	120	20	33.2	100
20	51	400	40	86.4	400
50	111	2177	100	282.2	2500
70	145	3954	140	429.0	4900
100	212	8188	200	664.4	10000

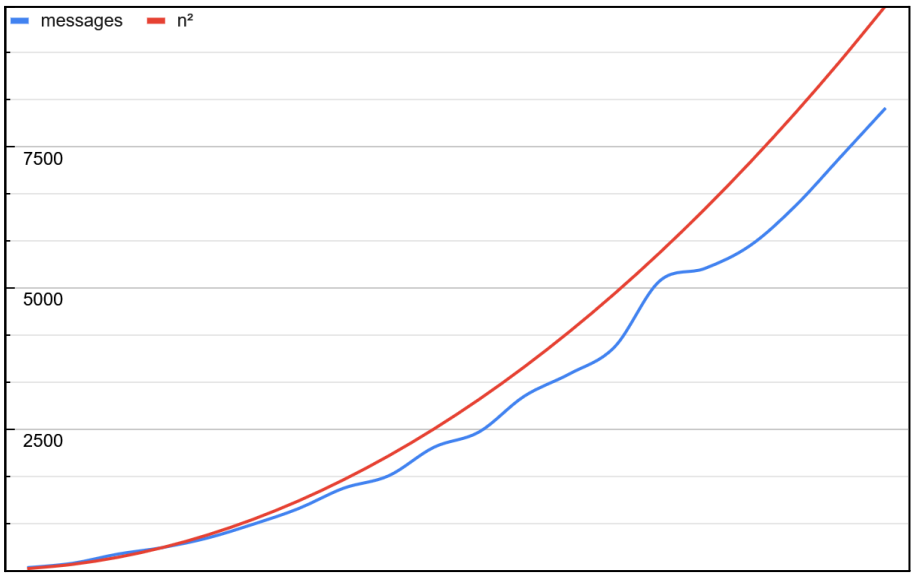
In the final simulation, a full table of results is provided which is also a great deal more interesting as it is a random ID assignment on a network of a ring of rings.

Rounds



The number of rounds hovers just above the $2n$ line which is what we expect. This is because the leader ID has to be transmitted $2n + 1$ times around the ring network. There are some slight bumps, which are due to the leader node-to-be-elected having varying wake rounds.

Messages



The number of messages stays under the worst case n^2 line which is what we expect. Near the start, wake round generation causes messages to just slip past the worst case line, but given larger n and the expected value of the number of messages, these are statistically improbable.

Analysis

The results match what is expected of the LCR algorithm. The exact expected and maximums are more difficult to calculate for a ring of rings network, but we don't necessarily need this. What is important is the growth rate (slope of the lines) and as long as those match up, we know that our LCR algorithm is performing as expected.

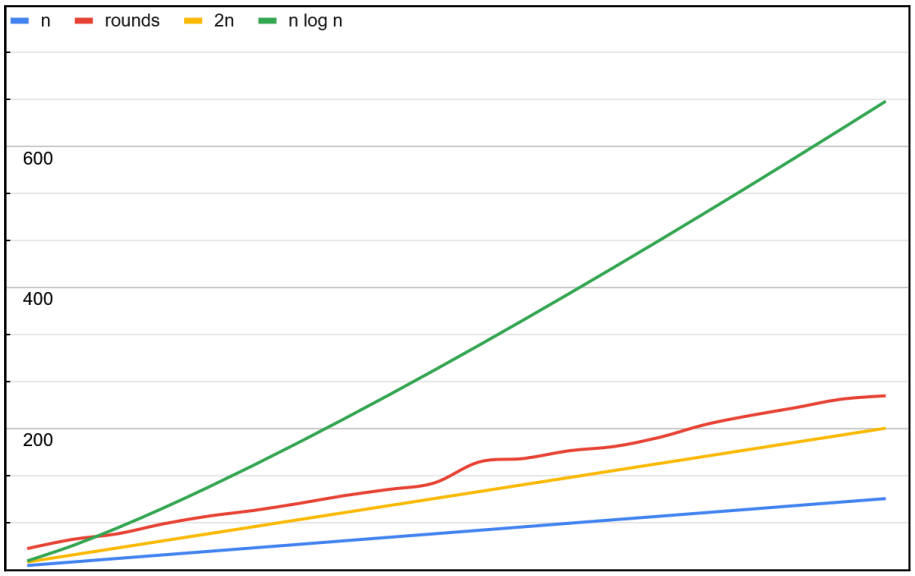
Ring of rings network tests

Ascending

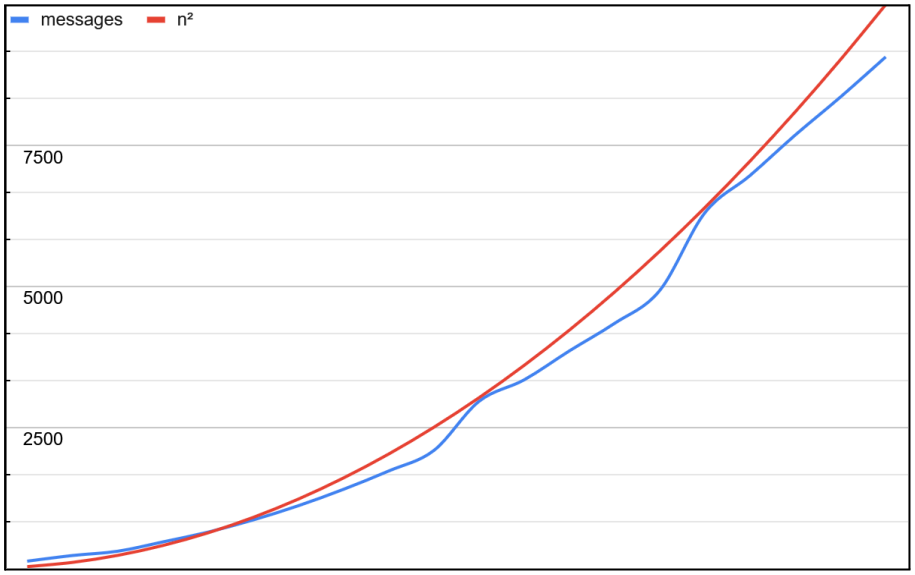
For $n = 5$ **Nodes** up to 100 in steps of 5 and $|k| = 3$ sub-rings up to 9 in steps of 3. Most results have been hidden for brevity but available on request.

n	|k|	avg. k	max k	Rounds	Msgs	2n	n log n	n²
10	3	5	5	42	224	20	33.2	100
55	6	8	13	152	2962	110	318.0	3025
100	9	11	19	246	9075	200	664.4	10000

Rounds



Messages

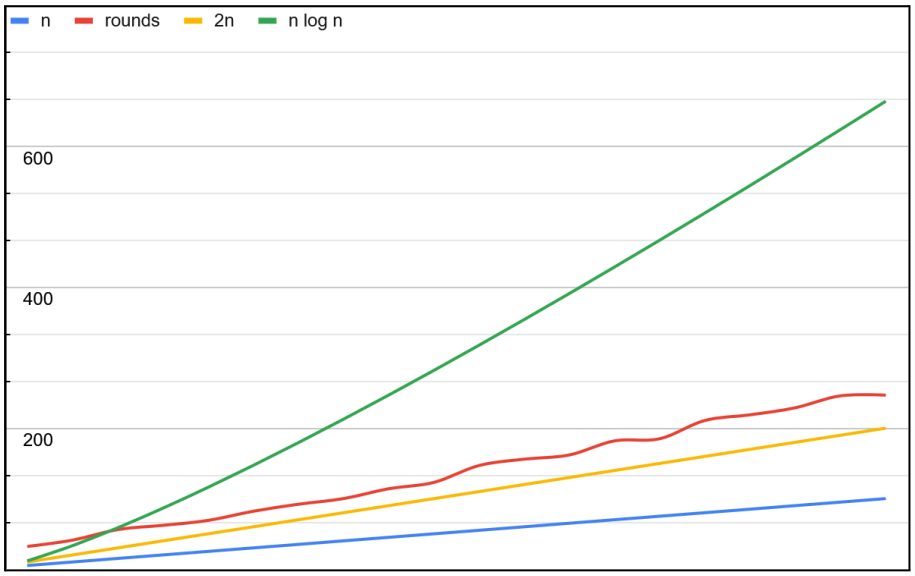


Descending

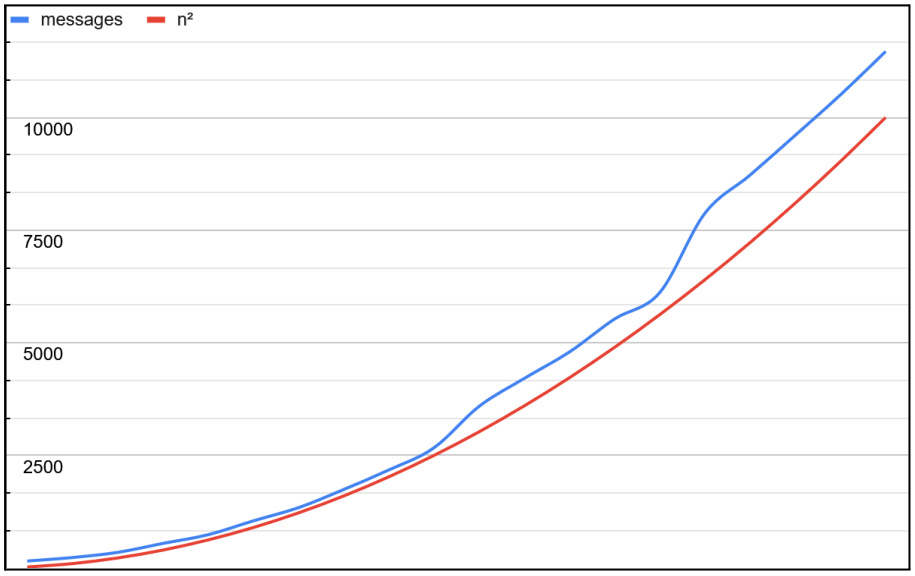
For $n = 5$ **Nodes** up to 100 in steps of 5 and $|k| = 3$ sub-rings up to 9 in steps of 3. Most results have been hidden for brevity but available on request.

n	|k|	avg. k	max k	Rounds	Msgs	2n	n log n	n²
10	3	5	5	41	233	20	33.2	100
55	6	8	13	147	3588	110	318.0	3025
100	9	11	19	247	11464	200	664.4	10000

Rounds



Messages



Random

Though not necessary I also did a random simulation. For $n = 5$ **Nodes** up to 100 in steps of 5 and $|k| = 3$ sub-rings up to 9 in steps of 3.

n	|k|	average k	max k	rounds	messages	2n	n log n	n²
5	3	5	5	34	177	10	11.6	25
10	3	5	5	43	221	20	33.2	100
15	3	5	5	49	306	30	58.6	225
20	3	5	5	63	474	40	86.4	400
25	3	5	5	71	674	50	116.1	625
30	3	5	5	86	892	60	147.2	900
35	3	5	5	94	1121	70	179.5	1225
40	3	5	5	105	1456	80	212.9	1600
45	3	5	5	112	1757	90	247.1	2025
50	3	5	5	123	2148	100	282.2	2500
55	6	8	13	144	2978	110	318.0	3025
60	6	8	13	153	3448	120	354.4	3600
65	6	8	13	169	3914	130	391.5	4225
70	6	8	13	171	4429	140	429.0	4900
75	6	8	13	196	5146	150	467.2	5625
80	9	11	19	208	6487	160	505.8	6400
85	9	11	19	216	7182	170	544.8	7225
90	9	11	19	227	7747	180	584.3	8100
95	9	11	19	237	8523	190	624.1	9025
100	9	11	19	246	9277	200	664.4	10000

Conclusions

Time and space complexity

All simulations follow the expected time and space growth. It is interesting to see how adding more sub-rings initially provides a sharp bump in complexity, but then levels out quickly to remain under the growth lines. Given large enough constants (sub-rings), these would rise above the lines but eventually fall back under given enough n **Nodes**.

Correctness

For each and every simulation, the LCR algorithm prints out the elected leader ID which was always the maximum ID that was generated => the correct leader was always elected